Física 2D e interacciones: construir la mecánica de juego

Ya podemos moverlo… pero atravesamos las Rocas

Presentación anterior

Input System

↓

`Vector2`

↓

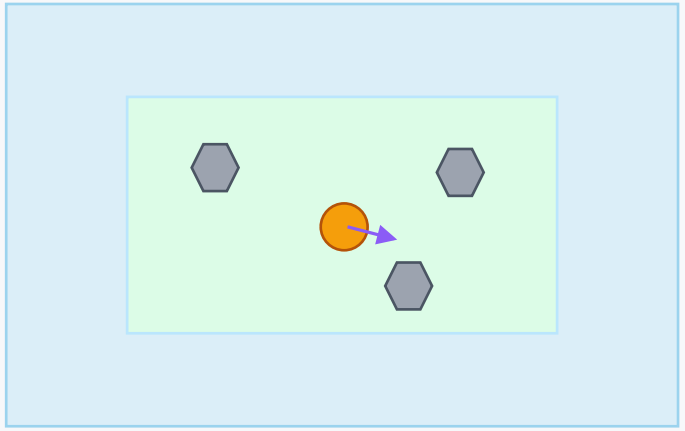
`Update` + `Time.deltaTime`

↓

`transform.position`

`Jugador` se mueve

pero …



Roca visible ≠ obstáculo físico

`Jugador` → atraviesa → `Roca`

Nuevo problema

¿Cómo hacemos que el motor gestione superficies, contactos e interacciones?

Movimiento programado
Nuestro código modifica explícitamente la posición.

Simulación física
Physics 2D participa en movimiento, contactos e interacciones.

Esta presentación: solo física 2D

`Rigidbody2D` · `Collider2D` · collision · trigger

Rigidbody2D: integrar el Jugador en Physics 2D

En los primeros pasos: **la funcionalidad se construye por composición**

`Jugador`

- Transform
- SpriteRenderer
- PlayerController
- PlayerInput
- Rigidbody2D

`Rigidbody2D`

Hace que el GameObject participe en la simulación Physics 2D.

Para nuestro `Jugador` cenital

- **Body Type → Dynamic**
- **Gravity Scale → 0**
- **Constraints → Freeze Rotation**

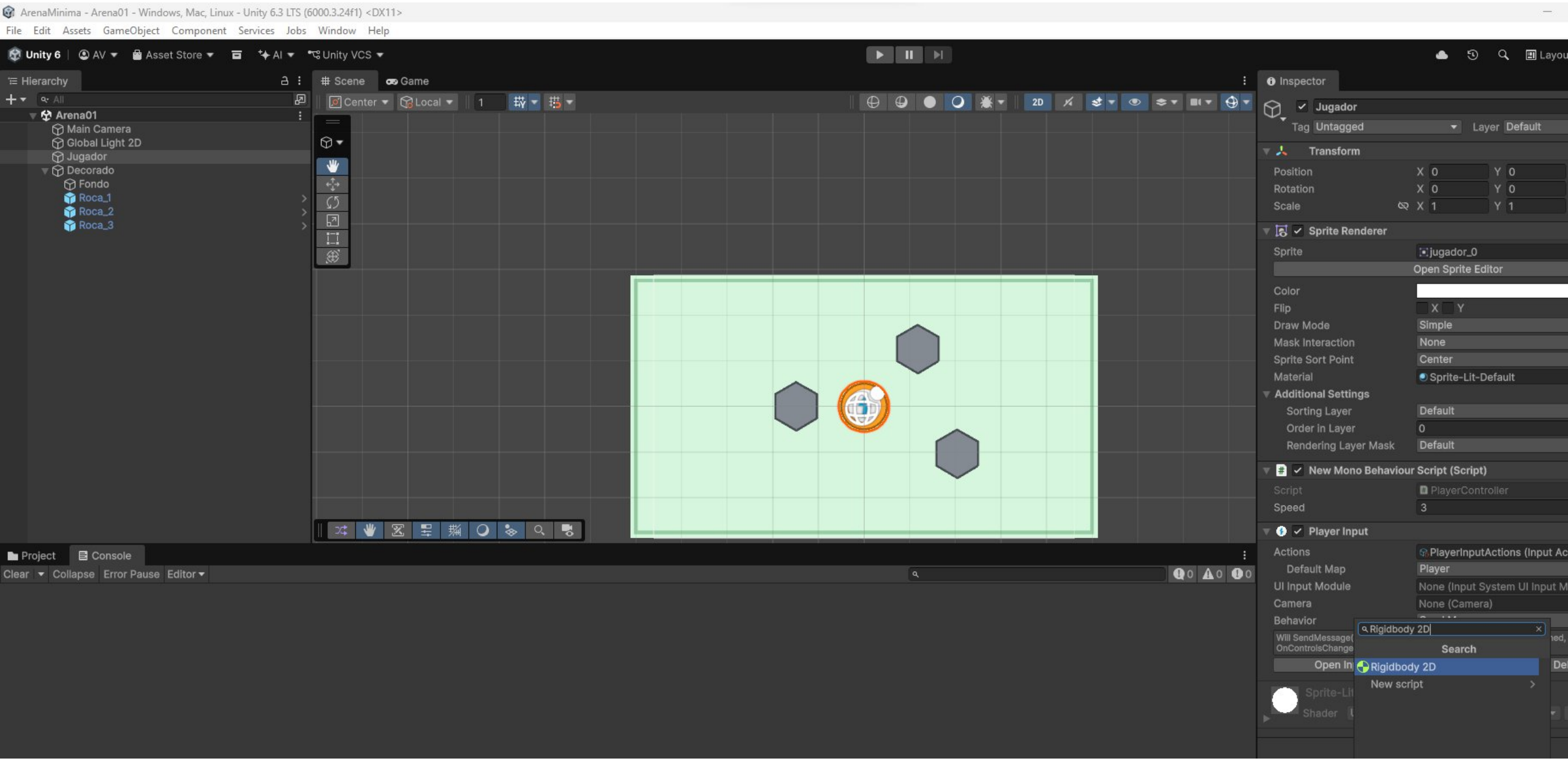
Gravedad 0
→ Y sigue siendo nuestro eje de movimiento

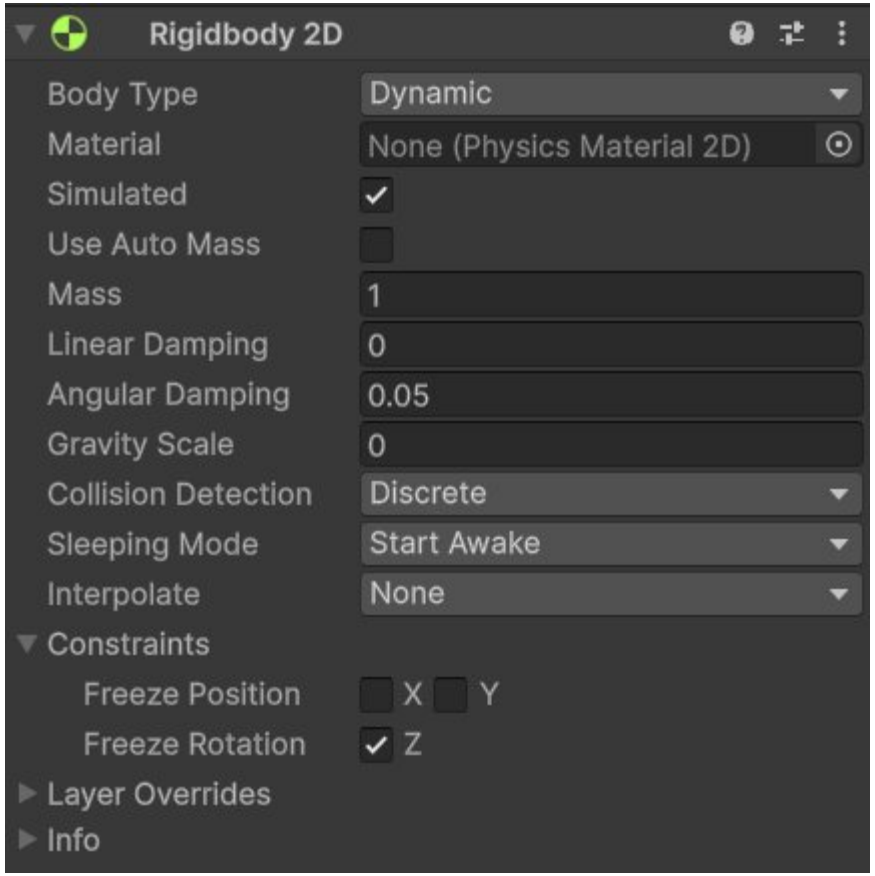
Freeze Rotation
→ evitamos giros no deseados en la mecánica

Todavía falta algo:
Rigidbody2D ≠ forma de contacto

Rigidbody2D: integrar el Jugador en Physics 2D

- Seleccionar el GameObject Jugador y añadirle un Component nuevo de tipo “RigidBody2D”.
- Editar las propiedades del componente añadido para que “Body Type” sea “Dynamic”, “Gravity Scale” sea “0” y el check “Freeze Rotation” en “Z” esté marcado.


ArenaMinima - Arena01 - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.24f1) <DX11>
File Edit Assets GameObject Component Services Jobs Window Help
Unity 6 AV Asset Store AI Unity VCS
Hierarchy
Arena01
Main Camera
Global Light 2D
Jugador
Decorado
Fondo
Roca_1
Roca_2
Roca_3
Scene
Game
Center
Local
2D
Inspector
Jugador
Tag Untagged
Layer Default
Transform
Position
Rotation
Scale
Sprite Renderer
Sprite jugador_0
Open Sprite Editor
Color
Flip
Draw Mode Simple
Mask Interaction None
Sprite Sort Point Center
Material Sprite-Lit-Default
Additional Settings
Sorting Layer Default
Order in Layer 0
Rendering Layer Mask Default
New Mono Behaviour Script (Script)
Script PlayerController
Speed 3
Player Input
Actions PlayerInputActions (Input Ac
Default Map Player
UI Input Module None (Input System UI Input M
Camera None (Camera)
Behavior
Rigidbody 2D
Search
Rigidbody 2D
New script
Project
Console
Clear
Collapse
Error Pause
Editor


Rigidbody 2D
Body Type
Dynamic
Material
None (Physics Material 2D)
Simulated
Use Auto Mass
Mass
1
Linear Damping
0
Angular Damping
0.05
Gravity Scale
0
Collision Detection
Discrete
Sleeping Mode
Start Awake
Interpolate
None
Constraints
Freeze Position
X
Y
Freeze Rotation
Z
Layer Overrides
Info

Dynamic, Kinematic y Static: elegir según el papel físico

Dynamic

Diseñado para moverse bajo simulación.

Puede responder a:

gravedad · fuerzas · contactos

Kinematic

Movimiento bajo control explícito.

No responde como Dynamic
a gravedad o fuerzas.

Static

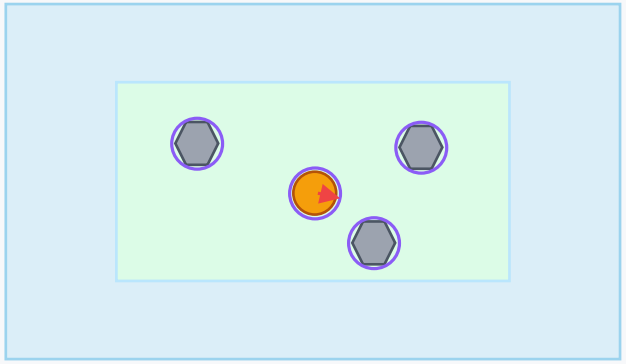
Diseñado para permanecer fijo
en la simulación.

Nuestra configuración

`Jugador`
Rigidbody2D Dynamic
+
Collider2D

→

`Rocas`
Collider2D
sin Rigidbody2D
Physics 2D las trata como estáticas



Body Type ≠ “se mueve / no se mueve”

Collider2D: lo visible y la forma física son cosas distintas

Representación visual

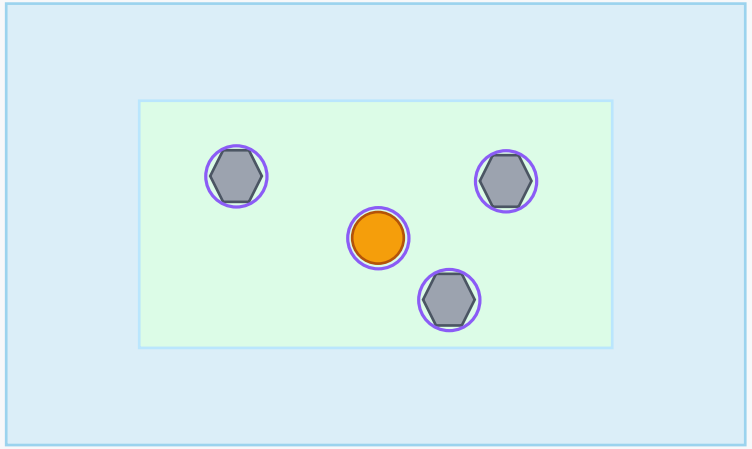
Sprite asset
↓
GameObject
+
`SpriteRenderer`

→ lo que vemos

Forma para Physics 2D

GameObject
+
`Collider2D`

→ forma usada para contactos y solapamientos



contornos púrpura = colliders

`Jugador`	`Roca.prefab`	`Meta`
sprite	sprite	sprite
+ CircleCollider2D	+ CircleCollider2D	+ BoxCollider2D más adelante

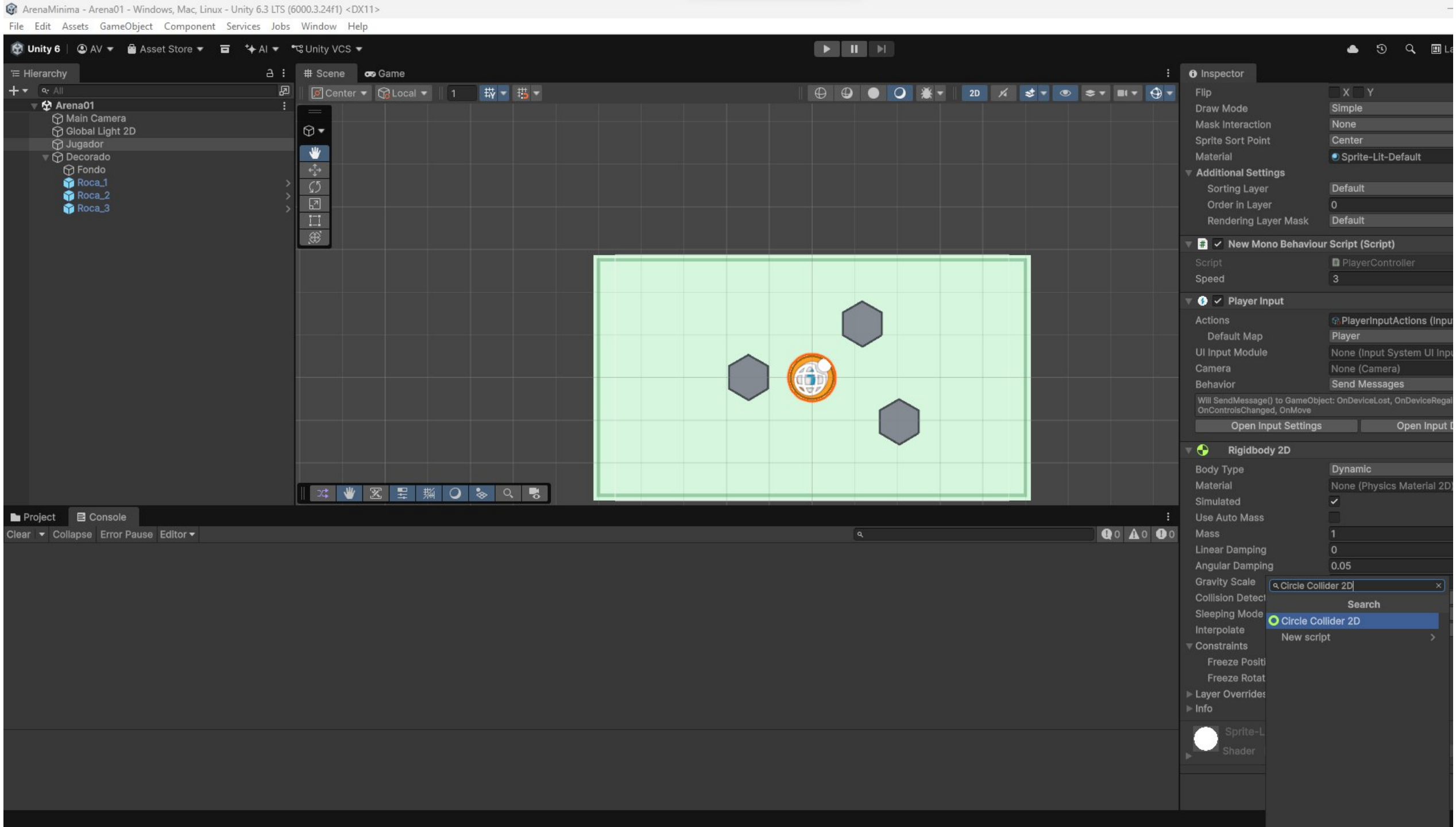
La forma debe aproximar suficientemente la mecánica.
No necesita seguir píxel a píxel el sprite.

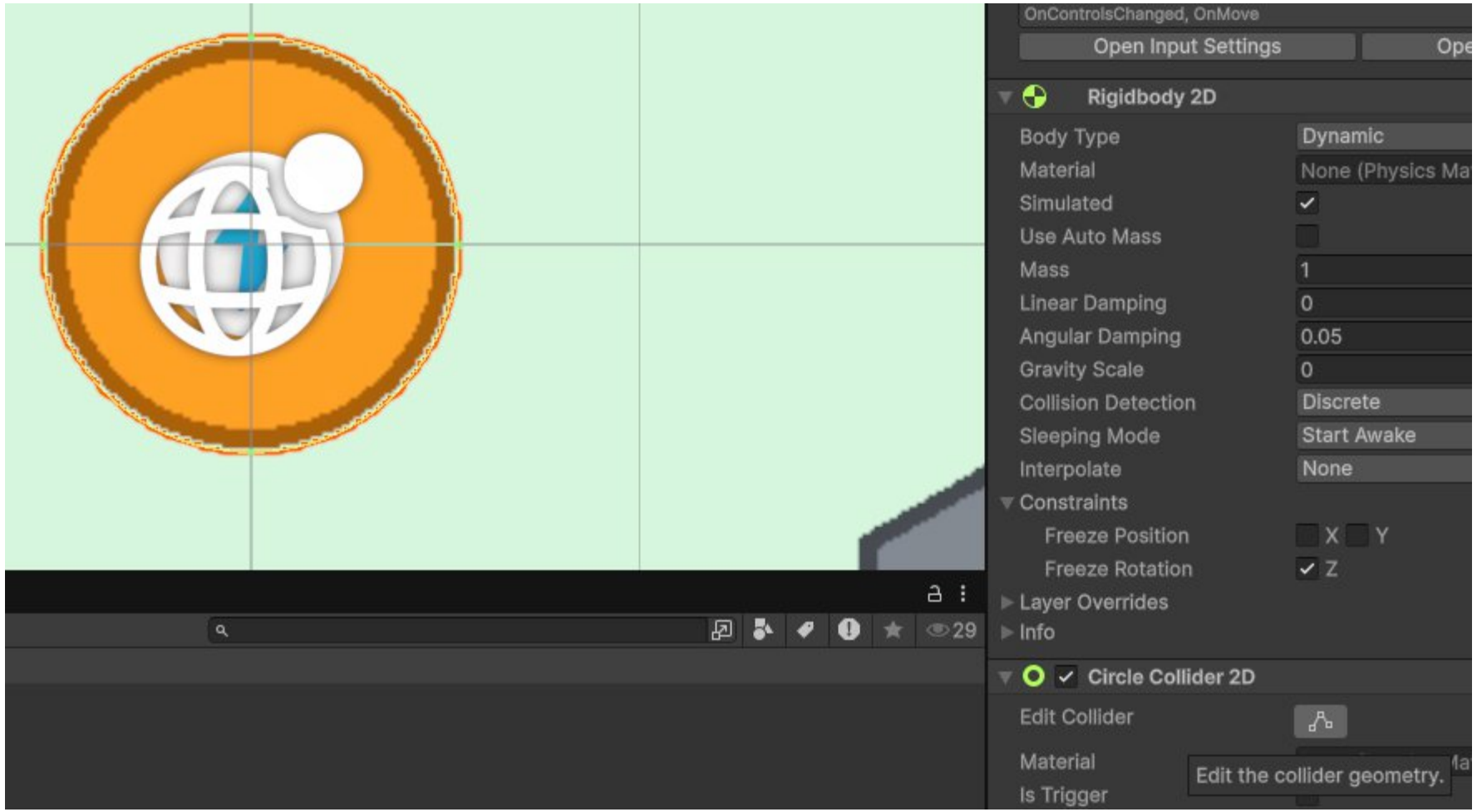
SpriteRenderer ≠ Collider2D

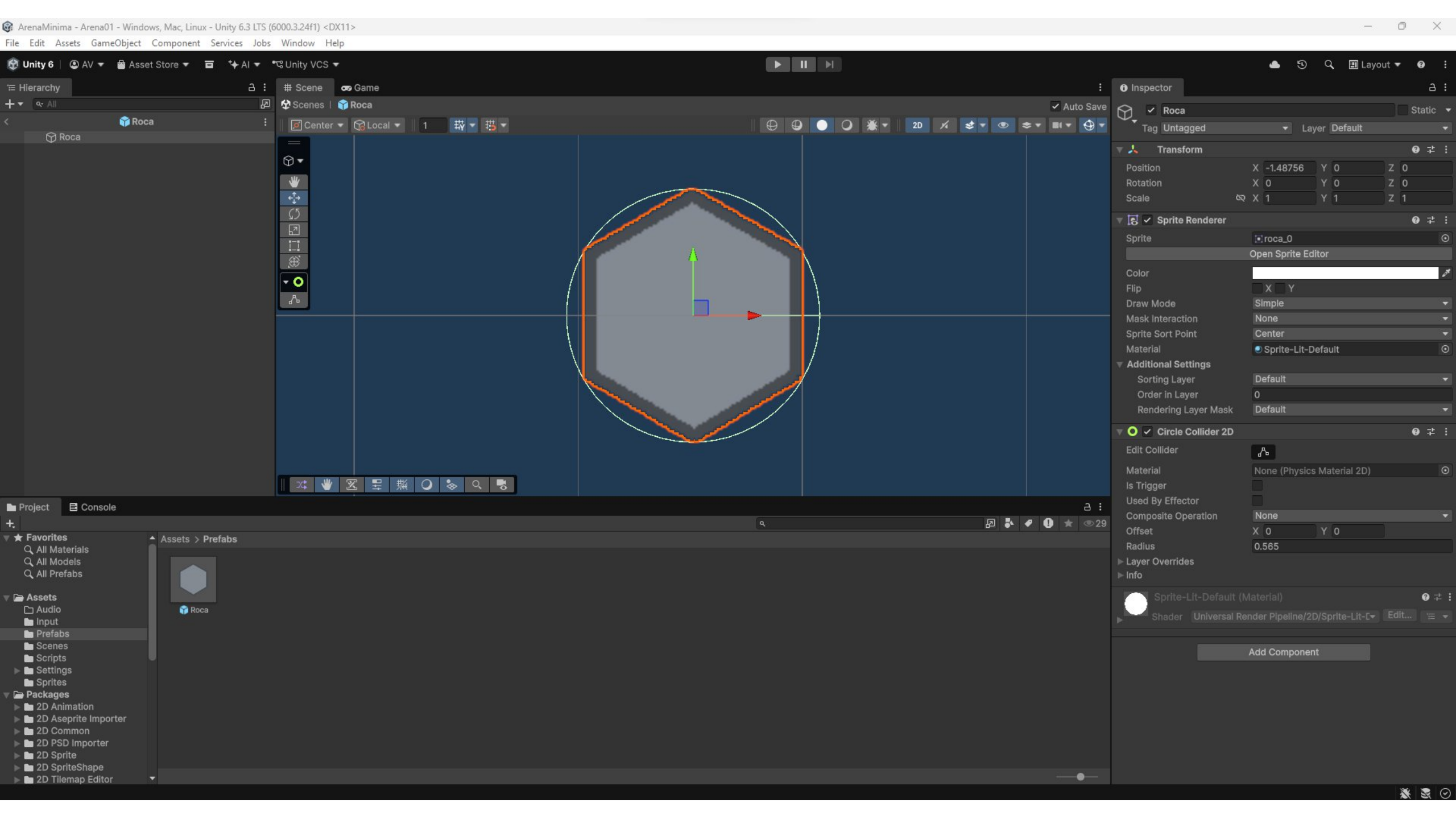
Collider2D ≠ Rigidbody2D

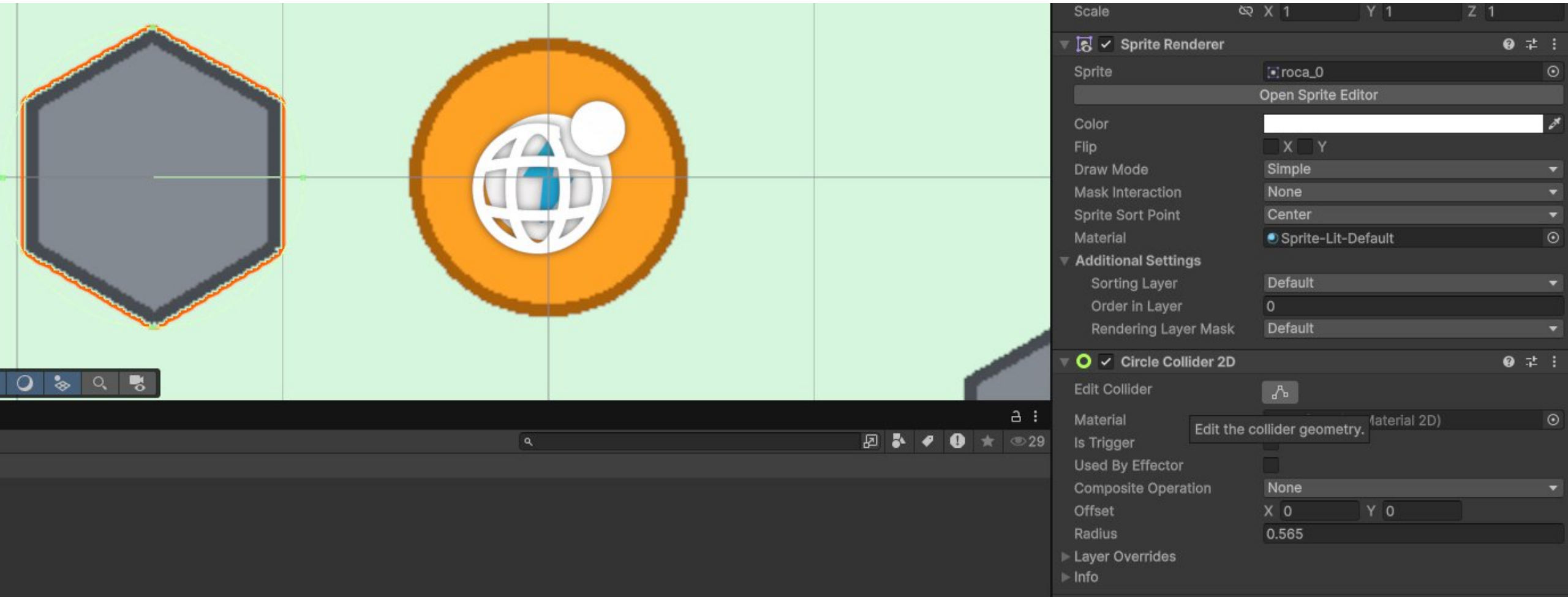
Collider2D: lo visible y la forma física son cosas distintas

- Seleccionar el GameObject Jugador y añadirle un Component nuevo de tipo “Circle Collider 2D”.
- Comprobar que el círculo encaja bien con el sprite de Jugador, pulsando en “Edit Collider” en el componente “Circle Collider 2D”.
- Seleccionar el Prefab Roca y añadirle un Component nuevo de tipo “Circle Collider 2D”.
- Comprobar que el círculo encaja de forma razonable con el sprite de Roca, pulsando en “Edit Collider” en el componente “Circle Collider 2D”.
- Guardar el Prefab Roca.
- Seleccionar un GameObject de roca y verificar que tiene el componente “Circle Collider 2D”.


ArenaMinima - Arena01 - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.24f1) <DX11>
File Edit Assets GameObject Component Services Jobs Window Help
Unity 6
Hierarchy
Arena01
Main Camera
Global Light 2D
Jugador
Decorado
Fondo
Roca_1
Roca_2
Roca_3
Scene
Game
Inspector
Flip
Draw Mode
Simple
Mask Interaction
None
Sprite Sort Point
Center
Material
Sprite-Lit-Default
Additional Settings
Sorting Layer
Default
Order in Layer
Rendering Layer Mask
New Mono Behaviour Script (Script)
Script
PlayerController
Speed
Player Input
Actions
Default Map
Player
UI Input Module
Camera
Behavior
Send Messages
Open Input Settings
Rigidbody 2D
Body Type
Dynamic
Material
Simulated
Use Auto Mass
Mass
Linear Damping
Angular Damping
0.05
Gravity Scale
Constraints
Layer Overrides
Info
Project
Console
Clear
Collapse
Error Pause
Editor
Circle Collider 2D
Search
Circle Collider 2D
New script


OnControlsChanged, OnMove
Open Input Settings
Rigidbody 2D
Body Type
Dynamic
Material
None (Physics Ma
Simulated
Use Auto Mass
Mass
1
Linear Damping
0
Angular Damping
0.05
Gravity Scale
0
Collision Detection
Discrete
Sleeping Mode
Start Awake
Interpolate
None
Constraints
Freeze Position
X
Y
Freeze Rotation
Z
Layer Overrides
Info
29
Circle Collider 2D
Edit Collider
Material
Is Trigger
Edit the collider geometry.


ArenaMinima - Arena01 - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.24f1) <DX11>
File Edit Assets GameObject Component Services Jobs Window Help
Unity 6 AV Asset Store AI Unity VCS Layout
Hierarchy
All
Roca
Roca
Scene Game
Scenes Roca
Auto Save
Center Local 1 2D
Inspector
Roca
Static
Tag Untagged Layer Default
Transform
Position X -1.48756 Y 0 Z 0
Rotation X 0 Y 0 Z 0
Scale X 1 Y 1 Z 1
Sprite Renderer
Sprite roca_0
Open Sprite Editor
Color
Flip X Y
Draw Mode Simple
Mask Interaction None
Sprite Sort Point Center
Material Sprite-Lit-Default
Additional Settings
Sorting Layer Default
Order in Layer 0
Rendering Layer Mask Default
Circle Collider 2D
Edit Collider
Material None (Physics Material 2D)
Is Trigger
Used By Effector
Composite Operation None
Offset X 0 Y 0
Radius 0.565
Layer Overrides
Info
Sprite-Lit-Default (Material)
Shader Universal Render Pipeline/2D/Sprite-Lit-D Edit...
Add Component
Project Console
Favorites
All Materials
All Models
All Prefabs
Assets
Audio
Input
Prefabs
Scenes
Scripts
Settings
Sprites
Packages
2D Animation
2D Aseprite Importer
2D Common
2D PSD Importer
2D Sprite
2D SpriteShape
2D Tilemap Editor
Assets > Prefabs
Roca


Scale
X 1
Y 1
Z 1
Sprite Renderer
Sprite
roca_0
Open Sprite Editor
Color
Flip
X
Y
Draw Mode
Simple
Mask Interaction
None
Sprite Sort Point
Center
Material
Sprite-Lit-Default
Additional Settings
Sorting Layer
Default
Order in Layer
0
Rendering Layer Mask
Default
Circle Collider 2D
Edit Collider
Material
Edit the collider geometry.
Is Trigger
Used By Effector
Composite Operation
None
Offset
X 0
Y 0
Radius
0.565
Layer Overrides
Info
29

P2 → P3: conservamos la entrada; cambia cómo movemos

Se mantiene

- Input System
- acción `Move`
- `OnMove(InputValue)`
- `moveInput : Vector2`
- +
- `speed`

En la presentación anterior

- `Update()`
- `transform.position`

cumplió el objetivo de la presentación anterior

En esta presentación

- `FixedUpdate()`
- `Rigidbody2D`
- `linearVelocity`

Una única ruta controla ahora el movimiento

✕ mover arbitrariamente por Transform y por Rigidbody2D a la vez

✓✓ entrada almacenada
→ ciclo físico
→ Rigidbody2D

`FixedUpdate ≠ Update “más preciso”`

PlayerController: ahora mueve el Rigidbody2D

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    [SerializeField] private float speed = 3f;

    private Vector2 moveInput;
    private Rigidbody2D rb;

    private void Start()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    public void OnMove(InputValue value)
    {
        moveInput = value.Get<Vector2>();
    }

    private void FixedUpdate()
    {
        rb.linearVelocity = moveInput * speed;
    }
}
```

`GetComponent<Rigidbody2D>()`

→ obtiene una referencia al Component existente
→ no crea el Component

`linearVelocity`

→ velocidad en unidades de mundo / segundo

Aquí no multiplicamos por `Time.deltaTime`.

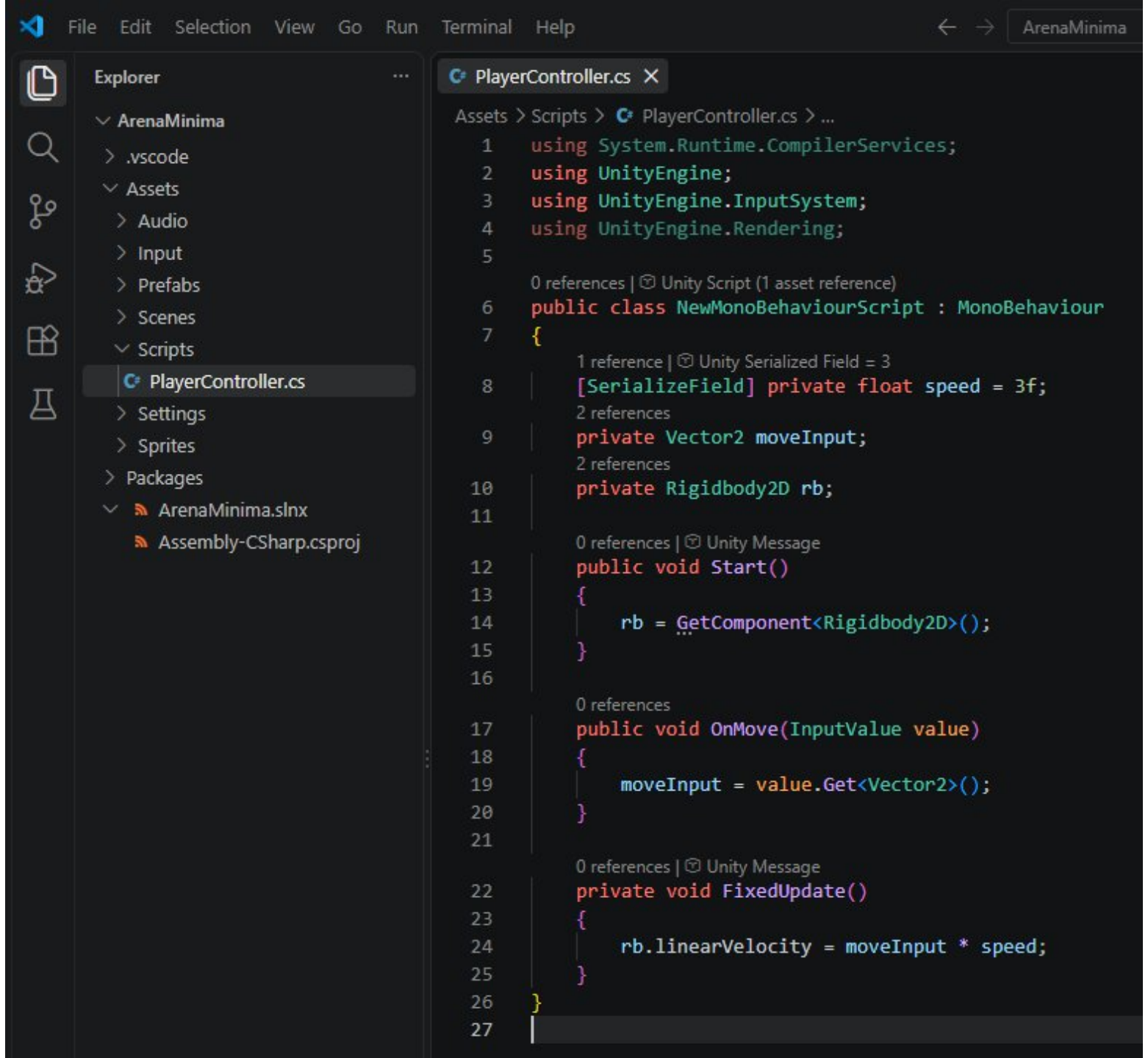
`moveInput` × `speed`
↓
`linearVelocity`

Presentación anterior
velocidad × tiempo
→ desplazamiento

Esta presentación asignamos velocidad al Rigidbody2D

PlayerController: ahora mueve el Rigidbody2D

- Abrir el script PlayerController.
- Modificar el código C# conforme a la imagen.
- Guardar y cerrar VS Code.
- Volver a Unity y esperar a que compile.
- Comprobar en Unity que no hay errores de compilación.
- Entrar en Play Mode, mover al jugador y comprobar que ahora choca con las piedras.
- Volver al Edit Mode.


File Edit Selection View Go Run Terminal Help
ArenaMinima
Explorer
ArenaMinima
.vscode
Assets
Audio
Input
Prefabs
Scenes
Scripts
PlayerController.cs
Settings
Sprites
Packages
ArenaMinima.slnx
Assembly-CSharp.csproj
PlayerController.cs
Assets > Scripts > PlayerController.cs > ...
using System.Runtime.CompilerServices;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.Rendering;
0 references | Unity Script (1 asset reference)
public class NewMonoBehaviourScript : MonoBehaviour
{
1 reference | Unity Serialized Field = 3
[SerializeField] private float speed = 3f;
2 references
private Vector2 moveInput;
2 references
private Rigidbody2D rb;
0 references | Unity Message
public void Start()
{
rb = GetComponent<Rigidbody2D>();
}
0 references
public void OnMove(InputValue value)
{
moveInput = value.Get<Vector2>();
}
0 references | Unity Message
private void FixedUpdate()
{
rb.linearVelocity = moveInput * speed;
}
}

La Roca ya bloquea: el contacto también puede generar un callback

Configuración

`Jugador`

Dynamic Rigidbody2D

+ CircleCollider2D

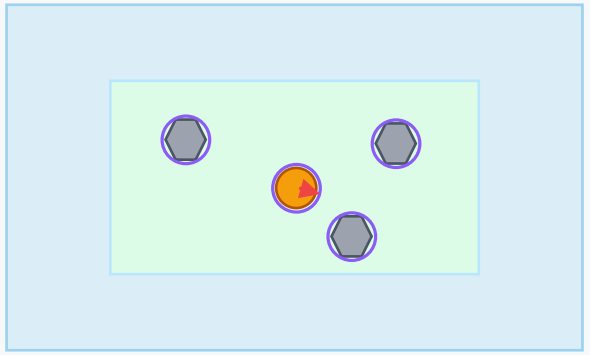
`Roca`

CircleCollider2D

sin Rigidbody2D
→ collider estático

Respuesta física

el Jugador no atraviesa la Roca



Reacción desde código

```
private void OnCollisionEnter2D(Collision2D collision)
{
    Debug.Log(
        "Contacto con: " + collision.gameObject.name
    );
}
```

`OnCollisionEnter2D`

→ Unity lo invoca al comenzar el contacto 2D

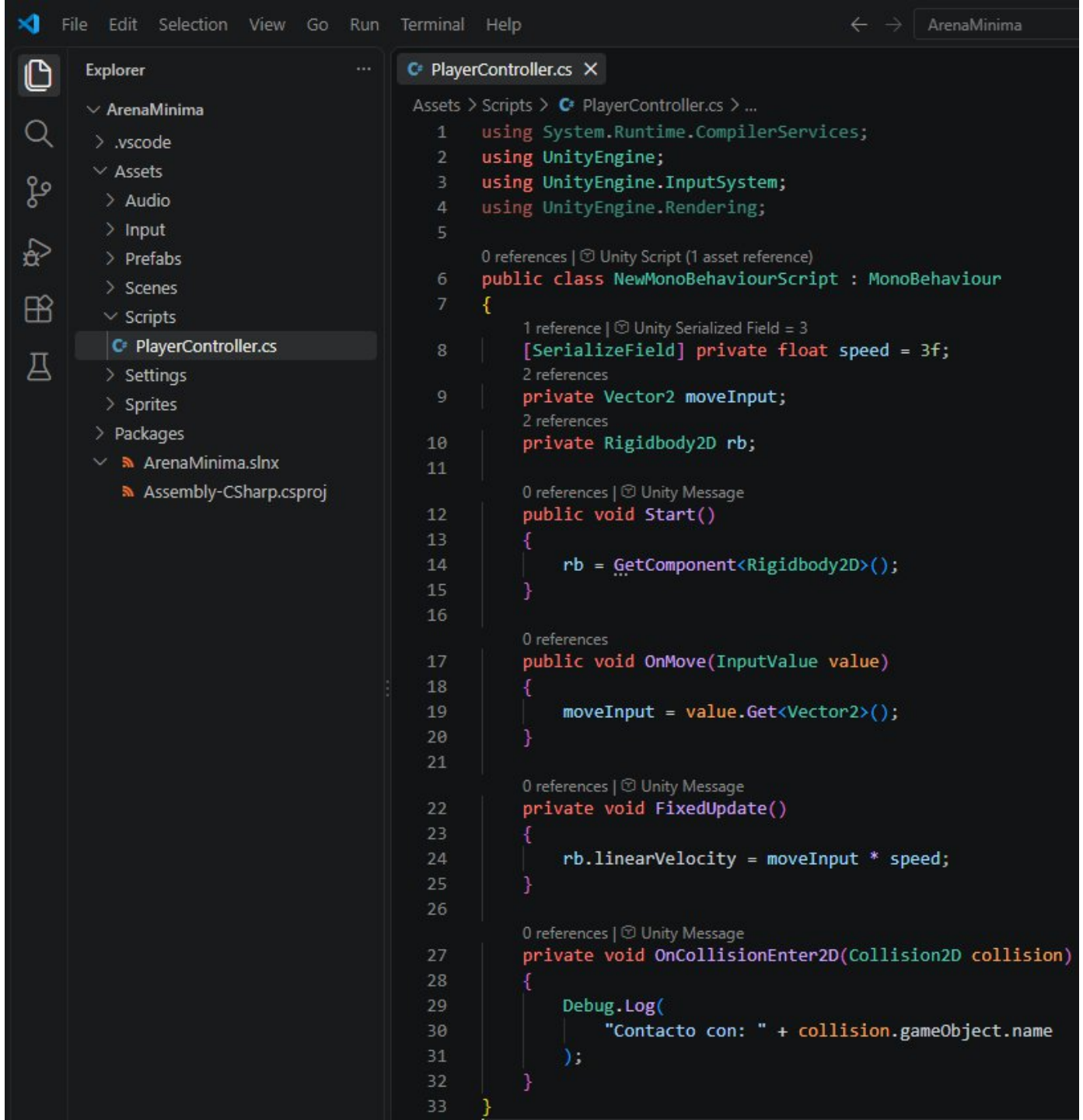
`Collision2D`

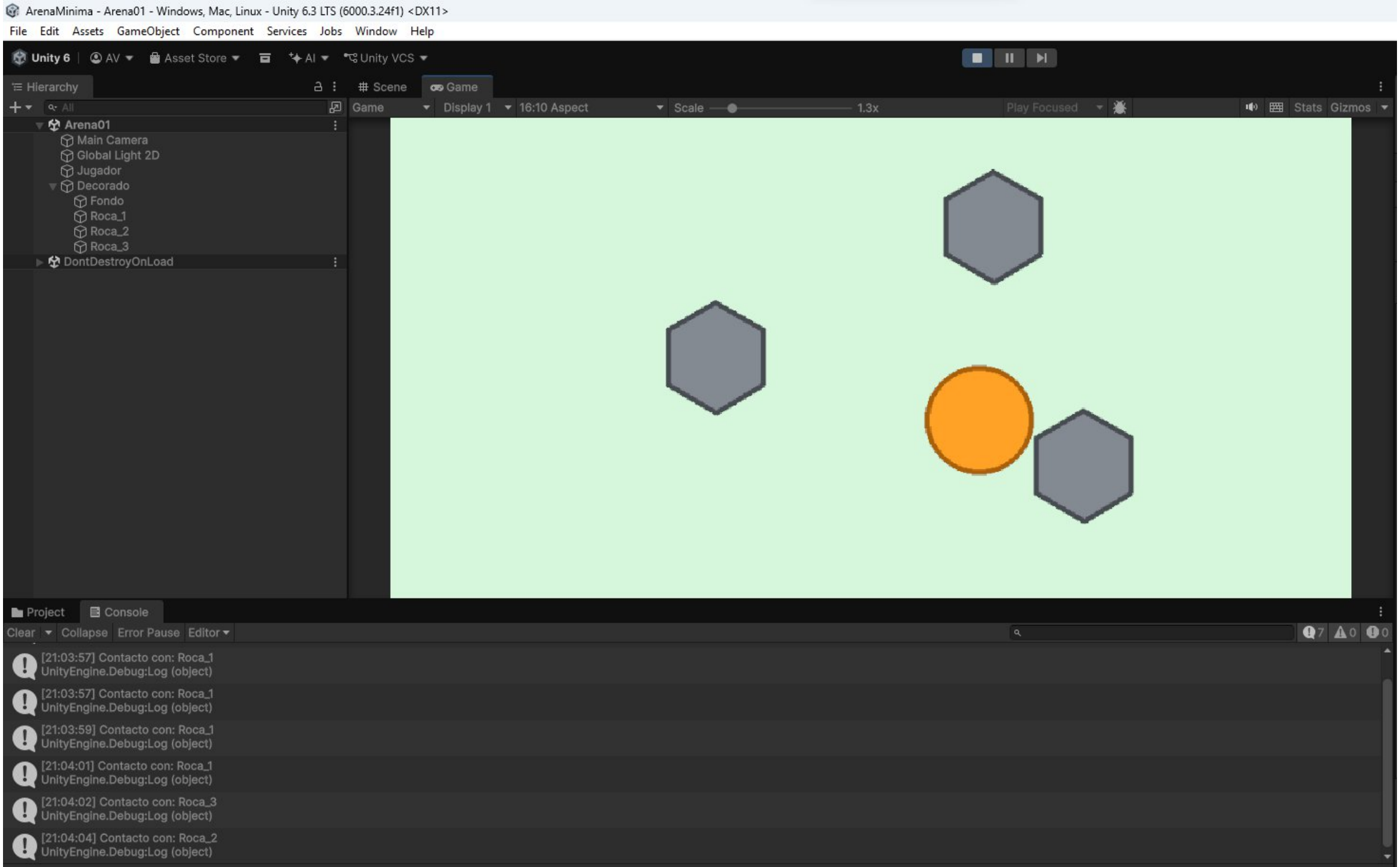
→ información sobre esa colisión

El callback no es lo que hace sólida a la Roca.

PlayerController: ahora mueve el Rigidbody2D

- Abrir el script PlayerController.
- Modificar el código C# conforme a la imagen.
- Guardar y cerrar VS Code.
- Volver a Unity y esperar a que compile.
- Comprobar en Unity que no hay errores de compilación.
- Entrar en Play Mode, chocar al jugador con alguna piedra y comprobar que en Console nos sale el mensaje correspondiente.
- Volver al Edit Mode.


File Edit Selection View Go Run Terminal Help
ArenaMinima
Explorer
ArenaMinima
.vscode
Assets
Audio
Input
Prefabs
Scenes
Scripts
PlayerController.cs
Settings
Sprites
Packages
ArenaMinima.slnx
Assembly-CSharp.csproj
PlayerController.cs
Assets > Scripts > PlayerController.cs > ...
using System.Runtime.CompilerServices;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.Rendering;
0 references | Unity Script (1 asset reference)
public class NewMonoBehaviourScript : MonoBehaviour
{
1 reference | Unity Serialized Field = 3
[SerializeField] private float speed = 3f;
2 references
private Vector2 moveInput;
2 references
private Rigidbody2D rb;
0 references | Unity Message
public void Start()
{
rb = GetComponent<Rigidbody2D>();
}
0 references
public void OnMove(InputValue value)
{
moveInput = value.Get<Vector2>();
}
0 references | Unity Message
private void FixedUpdate()
{
rb.linearVelocity = moveInput * speed;
}
0 references | Unity Message
private void OnCollisionEnter2D(Collision2D collision)
{
Debug.Log(
"Contacto con: " + collision.gameObject.name
);
}
}


ArenaMinima - Arena01 - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.24f1) <DX11>
File Edit Assets GameObject Component Services Jobs Window Help
Unity 6
AV
Asset Store
AI
Unity VCS
Hierarchy
Scene
Game
All
Arena01
Main Camera
Global Light 2D
Jugador
Decorado
Fondo
Roca_1
Roca_2
Roca_3
DontDestroyOnLoad
Game
Display 1
16:10 Aspect
Scale
1.3x
Play Focused
Stats
Gizmos
Project
Console
Clear
Collapse
Error Pause
Editor
[21:03:57] Contacto con: Roca_1
UnityEngine.Debug:Log (object)
[21:03:57] Contacto con: Roca_1
UnityEngine.Debug:Log (object)
[21:03:59] Contacto con: Roca_1
UnityEngine.Debug:Log (object)
[21:04:01] Contacto con: Roca_1
UnityEngine.Debug:Log (object)
[21:04:02] Contacto con: Roca_3
UnityEngine.Debug:Log (object)
[21:04:04] Contacto con: Roca_2
UnityEngine.Debug:Log (object)

Collision y Trigger resuelven problemas distintos

Obstáculo sólido

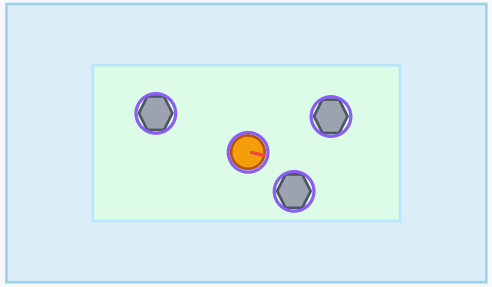
`Roca`

`CircleCollider2D`

Is Trigger = off

`Jugador` → contacto → bloqueo

callback posible:
`OnCollisionEnter2D`



Zona de interacción

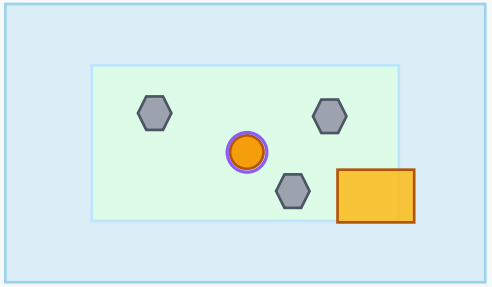
`Meta`

`BoxCollider2D`

Is Trigger = on

`Jugador` → solapamiento → atraviesa la zona

callback posible:
`OnTriggerEnter2D`



Para nuestra `Meta`

`Jugador`

- Dynamic Rigidbody2D
- + CircleCollider2D

`Meta`

- BoxCollider2D
- Is Trigger = on

Ambos pertenecen a Physics 2D.

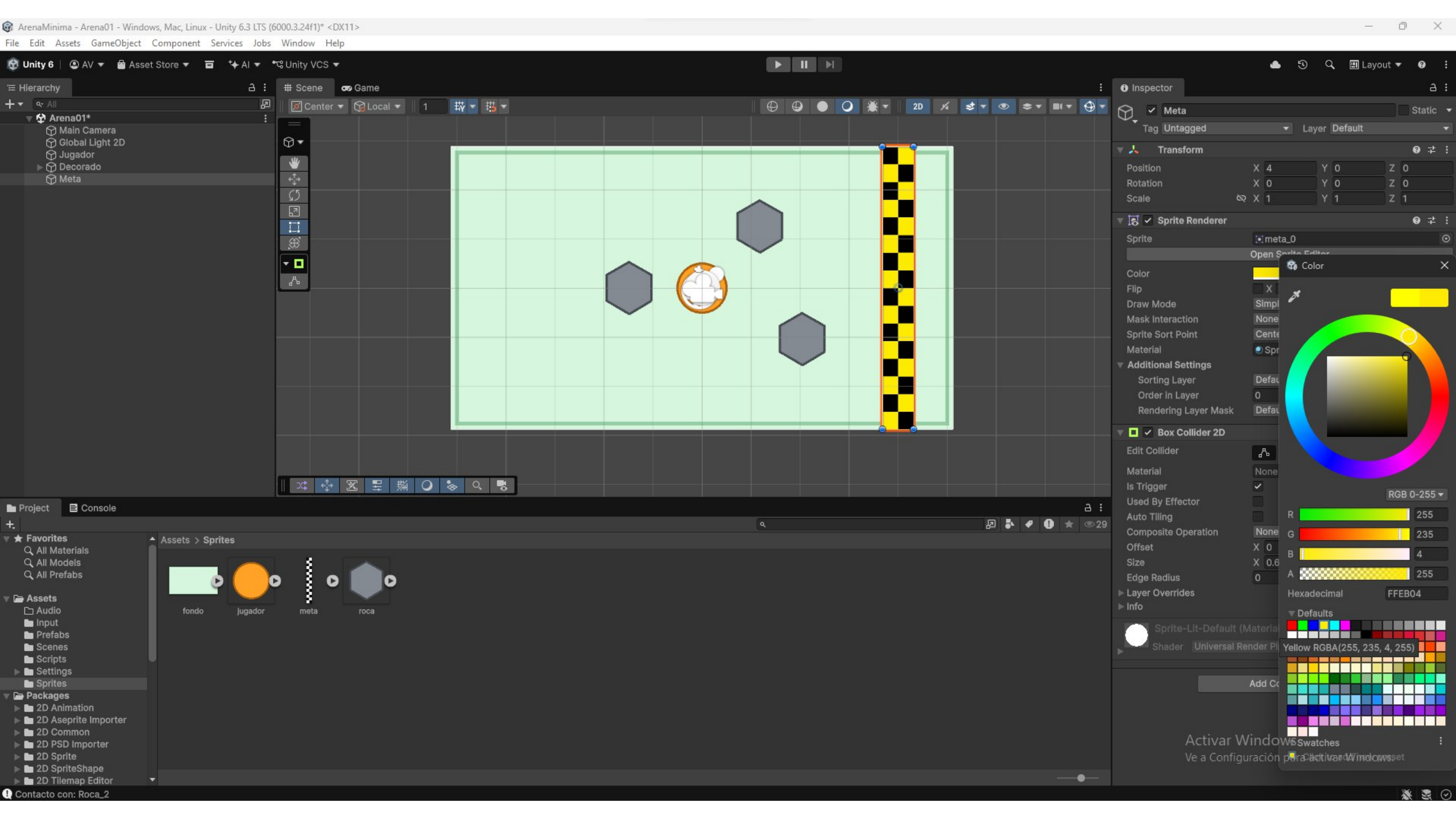
Trigger ≠ objeto invisible
Trigger ≠ Collider «sin física»

Collision y Trigger resuelven problemas distintos

- Creamos un GameObject vacío, lo renombramos “Meta” le agregamos un Component Sprite Renderer y en dicho componente le damos color amarillo.
- Nos descargamos “Meta.png” y lo importamos como Sprite a la carpeta Assets\Sprites vía Unity Editor.
- Añadimos al Component Sprite Renderer de nuestro GameObject Meta el sprite que acabamos de importar.
- Colócamos el GameObject “Meta” en alguna zona libre del GameObject Fondo.

Collision y Trigger resuelven problemas distintos

- Al GameObject “Meta” le añadimos un Component BoxCollider2D y en dicho componente marcamos “Is Trigger”.
- En el GameObject Jugador, en su Component Sprite Redender, ponemos el valor 1 en “Order in Layer”.
- Entramos al Play Mode y comprobamos que Jugador puede atravesar Meta y que el sprite de Jugador se superpone al sprite de Meta cuando se solapan.
- Volvemos al Edit Mode.


ArenaMinima - Arena01 - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.24f1)* <DX11>
File Edit Assets GameObject Component Services Jobs Window Help
Unity 6
Hierarchy
Arena01*
Main Camera
Global Light 2D
Jugador
Decorado
Meta
Scene
Game
Inspector
Meta
Tag Untagged
Layer Default
Transform
Position
Rotation
Scale
Sprite Renderer
Sprite meta_0
Color
Flip
Draw Mode
Mask Interaction
Sprite Sort Point
Material
Additional Settings
Sorting Layer
Order in Layer
Rendering Layer Mask
Box Collider 2D
Edit Collider
Material
Is Trigger
Used By Effector
Auto Tiling
Composite Operation
Offset
Size
Edge Radius
Layer Overrides
Info
Color
RGB 0-255
R 255
G 235
B 4
A 255
Hexadecimal FFEB04
Defaults
Yellow RGBA(255, 235, 4, 255)
Swatches
Project
Console
Favorites
All Materials
All Models
All Prefabs
Assets
Audio
Input
Prefabs
Scenes
Scripts
Settings
Sprites
Packages
2D Animation
2D Aseprite Importer
2D Common
2D PSD Importer
2D Sprite
2D SpriteShape
2D Tilemap Editor
Assets > Sprites
fondo
jugador
meta
roca
Activar Windows
Ve a Configuración para activar Windows.
Contacto con: Roca_2

Del trigger a una regla: el evento cambia el estado

`Jugador` entra en `Meta` → `OnTriggerEnter2D` → `goalReached = true` → `Meta` cambia a verde → Console: `Meta alcanzada`

```
private bool goalReached = false;

private void OnTriggerEnter2D(Collider2D other)
{
    if (goalReached) return;

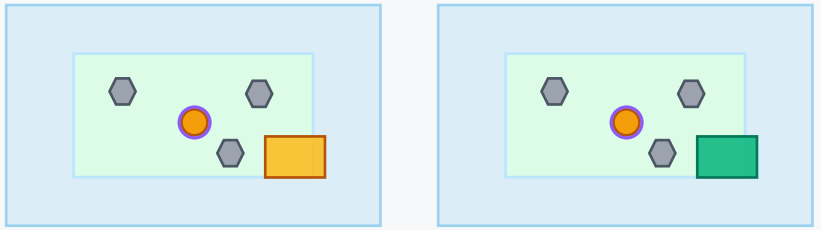
    goalReached = true;

    other.GetComponent<SpriteRenderer>().color =
        Color.green;

    Debug.Log("Meta alcanzada");
}
```

`Collider2D other`

→ el otro Collider2D implicado



antes

después

En esta escena
solo existe un trigger: `Meta`

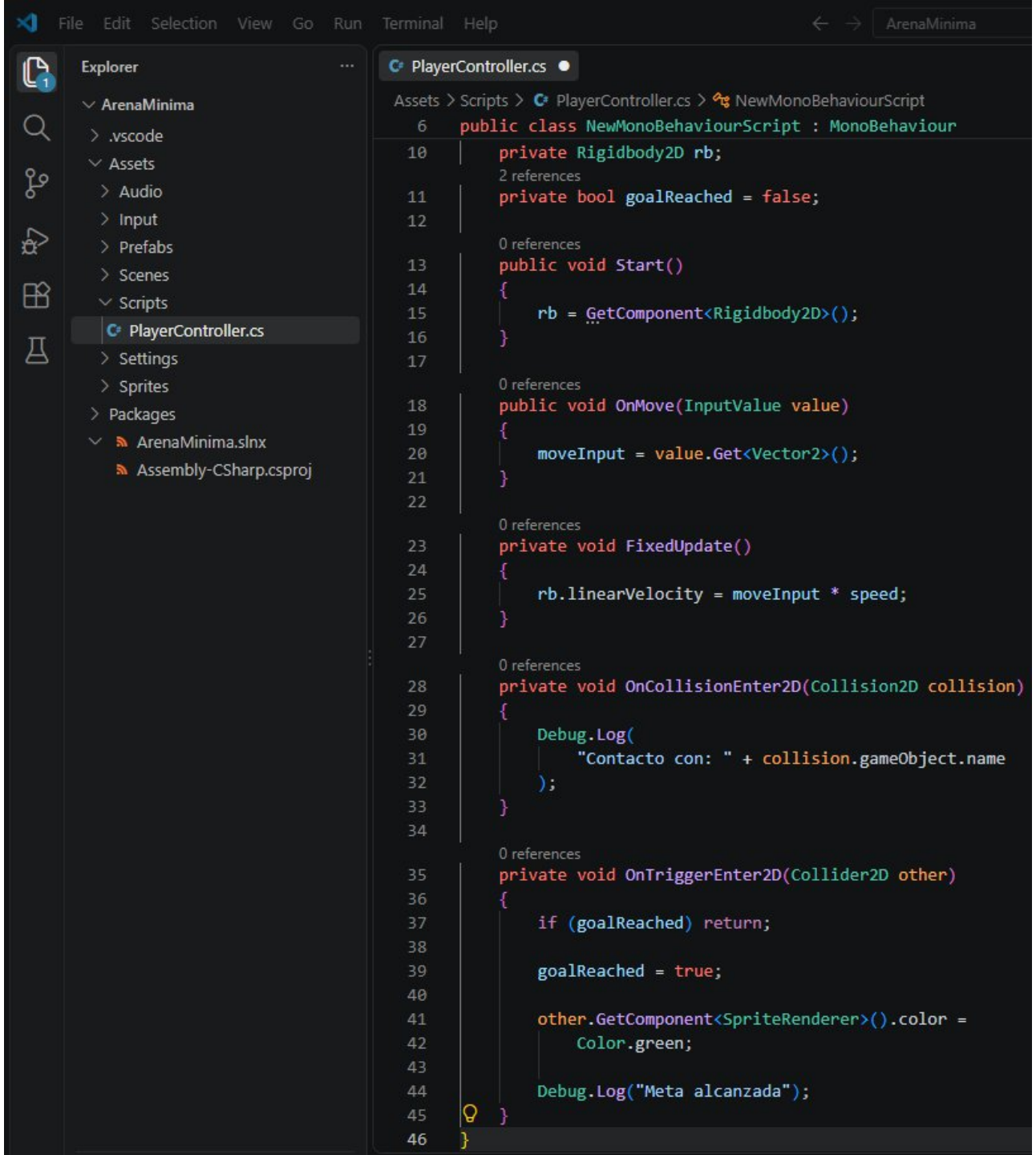
En la siguiente presentación comunicará este estado al jugador de forma adecuada.

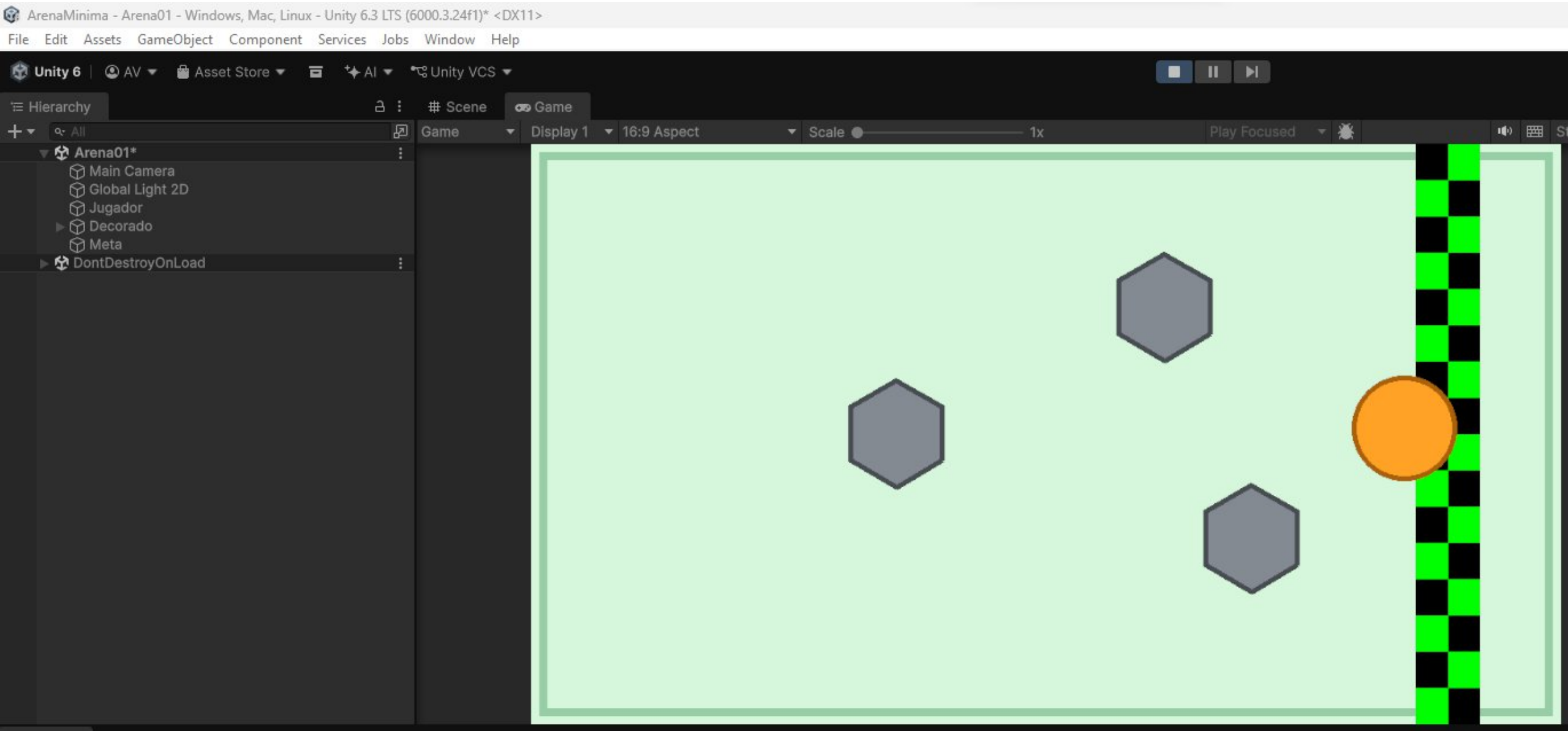
Del trigger a una regla: el evento cambia el estado

- Abrir el script PlayerController.
- Modificar el código C# conforme a la imagen.
- Guardar y cerrar VS Code.
- Volver a Unity y esperar a que compile.
- Comprobar en Unity que no hay errores de compilación.

Del trigger a una regla: el evento cambia el estado

- Entramos al Play Mode y comprobamos que cuando Jugador atraviesa Meta el color de meta cambia correspondientemente.
- Volvemos al Edit Mode.


File Edit Selection View Go Run Terminal Help
ArenaMinima
Explorer
ArenaMinima
.vscode
Assets
Audio
Input
Prefabs
Scenes
Scripts
PlayerController.cs
Settings
Sprites
Packages
ArenaMinima.slnx
Assembly-CSharp.csproj
PlayerController.cs
Assets > Scripts > PlayerController.cs > NewMonoBehaviourScript
public class NewMonoBehaviourScript : MonoBehaviour
private Rigidbody2D rb;
2 references
private bool goalReached = false;
0 references
public void Start()
{
rb = GetComponent<Rigidbody2D>();
}
0 references
public void OnMove(InputValue value)
{
moveInput = value.Get<Vector2>();
}
0 references
private void FixedUpdate()
{
rb.linearVelocity = moveInput * speed;
}
0 references
private void OnCollisionEnter2D(Collision2D collision)
{
Debug.Log(
"Contacto con: " + collision.gameObject.name
);
}
0 references
private void OnTriggerEnter2D(Collider2D other)
{
if (goalReached) return;
goalReached = true;
other.GetComponent<SpriteRenderer>().color =
Color.green;
Debug.Log("Meta alcanzada");
}
}


ArenaMinima - Arena01 - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.24f1)* <DX11>
File Edit Assets GameObject Component Services Jobs Window Help
Unity 6
AV
Asset Store
AI
Unity VCS
Hierarchy
Scene
Game
All
Arena01*
Main Camera
Global Light 2D
Jugador
Decorado
Meta
DontDestroyOnLoad
Game
Display 1
16:9 Aspect
Scale
1x
Play Focused

Probar la mecánica: caso normal y un límite sencillo

Probar una mecánica ≠ comprobar solo la última línea

1. Movimiento

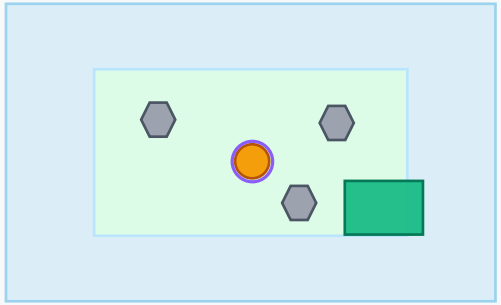
✓ WASD mueve `Jugador`

✓ sin entrada → velocidad 0

2. Obstáculos

✓ `Jugador` no atraviesa las Rocas

✓ aparece el callback de contacto



3. Meta: caso normal

✓ entrar en `Meta`
✓ `goalReached` pasa a `true`
✓ Meta cambia a verde
✓ Console: `Meta alcanzada`

4. Límite sencillo

salir de `Meta`
→ volver a entrar

✓ no se ejecuta otra vez la consecuencia
✓ no aparece otro `Meta alcanzada`

`goalReached == true`
→ no repetir consecuencia

Console

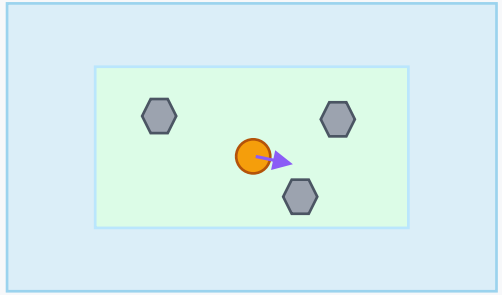
✓ sin errores recurrentes

modificar → probar → observar → diagnosticar → corregir → volver a probar

Ya tenemos una mecánica jugable mínima

Presentación anterior:

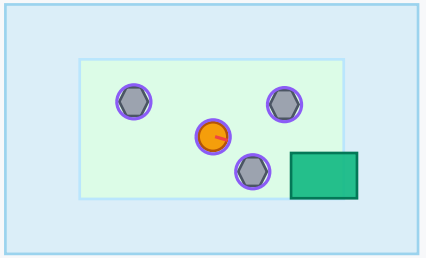
Input System
↓
movimiento mediante código
↓
Rocas atravesables



Esta presentación:

Input System
↓
`moveInput`
↓
`FixedUpdate`
↓
`Rigidbody2D`
+
`Collider2D`
↓
Rocas sólidas

trigger `Meta`
↓
`OnTriggerEnter2D`
↓
`goalReached`
↓
consecuencia jugable



`Jugador`

- ✓ sigue controlado por Input System
- ✓ participa en Physics 2D
- ✓ contacta con obstáculos
- ✓ activa una interacción significativa
- ✓ modifica el estado de la partida

La mecánica funciona.

Próxima presentación:

¿Cómo la convertimos en un juego comprensible, comprobado y entregable?